

INDEX

S. No	Title of the Experiment	Page No.
1	Install Hadoop and configure Hadoop Distributed File System (HDFS) in standalone or pseudo-distributed mode.	1
2	Demonstrate basic HDFS commands for file creation, upload, download, replication, and deletion operations.	
3	Develop a MapReduce program to perform Word Count analysis on a text dataset.	
4	Implement Hadoop Streaming using Python scripts for simple data processing tasks.	
5	Perform data storage and retrieval operations using Cassandra Query Language (CQL).	
6	Create and manipulate Key-Value and Document-oriented data models using NoSQL databases.	
7	Develop simple Pig Latin scripts to perform filtering, grouping, joining, and aggregation operations on datasets.	
8	Create Hive tables and execute HiveQL queries including DDL commands and simple analytical queries.	
9	Create data visualizations such as Line Charts, Bar Charts, Pie Charts, Scatter Plots, and Box Plots using Python libraries.	
10	Perform case study implementation involving installation and execution of Hive with sample datasets.	

SOFTWARE REQUIREMENTS:

1. Java JDK 8.0 and above
2. Apache Hadoop 2.7 +

Ex No.	01	Install Hadoop and configure Hadoop Distributed File System (HDFS) in standalone or pseudo-distributed mode.
Date :		

AIM:

To perform setting up and installing Hadoop in its three operating modes:

- Standalone
- Pseudo Distributed
- Fully Distributed

DESCRIPTION:

Hadoop is written in Java, so you will need to have Java installed on your machine, version 6 or later. Sun's JDK is the one most widely used with Hadoop, although others have been reported to work.

Hadoop runs on Unix and on Windows. Linux is the only supported production platform, but other flavors of Unix (including Mac OS X) can be used to run Hadoop for development. Windows is only supported as a development platform, and additionally requires Cygwin to run. During the Cygwin installation process, you should include the openssh package if you plan to run Hadoop in pseudo- distributed mode.

ALGORITHM:**STEPS INVOLVED IN INSTALLING HADOOP IN STANDALONE MODE:**

1. Command for installing ssh is “**sudo apt-get install ssh**”.
2. Command for key generation is **ssh-keygen -t rsa -P “ ”**.
3. Store the key into rsa.pub by using the command **cat \$HOME/.ssh/id_rsa.pub >> \$HOME/.ssh/authorized_keys**
4. Extract the java by using the command **tar xvfz jdk-8u60-linux-i586.tar.gz**.
5. Extract the eclipse by using the command **tar xvfz eclipse-jee-mars-R-linux-gtk.tar.gz**
6. Extract the hadoop by using the command **tar xvfz hadoop-2.7.1.tar.gz**
7. Extract the hadoop by using the command **tar xvfz hadoop-2.7.1.tar.gz**
8. Move the java to **/usr/lib/jvm/** and eclipse to **/opt/** paths. Configure the java path

in the eclipse.ini file

9. Export java path and hadoop path in ./bashrc
10. Check the installation successful or not by checking the java version and hadoop version
11. Check the hadoop instance in standalone mode working correctly or not by using an implicit hadoop jar file named as word count.
12. If the word count is displayed correctly in part-r-00000 file it means that standalone mode is installed successfully.

ALGORITHM:

STEPS INVOLVED IN INSTALLING HADOOP IN PSEUDO DISTRIBUTED MODE:

1. In order install pseudo distributed mode we need to configure the hadoop configuration files resides in the directory/home/lendi/hadoop-2.7.1/etc/hadoop.
2. First configure the hadoop-env.sh file by changing the java path.
3. Configure the core-site.xml which contains a property tag, it contains name and value. Name as fs.defaultFS and value as hdfs://localhost:9000
4. Configure hdfs-site.xml.
5. Configure yarn-site.xml.
6. Configure mapred-site.xml before configure the copy mapred-site.xml.template to mapred-site.xml.
7. Now format the name node by using command hdfs namenode –format.
8. Type the command start-dfs.sh, start-yarn.sh means that starts the daemons like Name Node, Data Node, SecondaryNameNode, Resource Manager, Node Manager.
9. Run JPS which views all daemons. Create a directory in the hadoop by using command hdfs dfs –mkdir /csedir and enter some data into lendi.txt using command nano lendi.txt and copy from local directory to hadoop using command hdfs dfs –copy FromLocal lendi.txt /csedir/ and run sample jar file word count to check whether pseudo distributed mode is working or not.
10. Display the contents of file by using command hdfs dfs –cat /newdir/part-r-00000.

FULLY DISTRIBUTED MODE:

INSTALLATION: ALGORITHM:

1. Stop all single node clusters

```
$stop-all.sh
```

2. Decide one as Name Node (Master) and remaining as Data Nodes (Slaves).

3. Copy public key to all three hosts to get a password less SSH access

```
$ssh-copy-id -I $HOME/.ssh/id_rsa.pub lendi@l5sys24
```

4. Configure all Configuration files, to name Master and Slave Nodes.

```
$cd $HADOOP_HOME/etc/hadoop
```

```
$nano core-site.xml
```

```
$ nano hdfs-site.xml
```

5. Add hostnames to file slaves and save it.

```
$ nano slaves
```

6. Configure \$ nano yarn-site.xml

7. Do in Master Node

```
$ hdfs namenode -format
```

```
$ start-dfs.sh
```

```
$start-yarn.sh
```

8. Format Name Node

9. Daemons Starting in Master and Slave Nodes

10. End

INPUT:

ubuntu @localhost> jps

OUTPUT:

Data node, name nodem Secondary name node,

NodeManager, Resource Manager

RESULT:

Hence Installation of Hadoop in different modes was studied.

Ex No.	2	DEMONSTRATE BASIC HDFS COMMANDS FOR FILE CREATION, UPLOAD, DOWNLOAD, REPLICATION, AND DELETION OPERATIONS.
Date		

AIM:

To implement the following file management tasks in Hadoop:

- Adding files and directories
- Retrieving files
- Deleting Files

DESCRIPTION:

HDFS is a scalable distributed file system designed to scale to peta bytes of data while running on top of the underlying file system of the operating system. HDFS keeps track of where the data resides in a network by associating the name of its rack (or network switch) with the dataset. This allows Hadoop to efficiently schedule tasks to those nodes that contain data, or which are nearest to it, optimizing bandwidth utilization. Hadoop provides a set of command line utilities that work similarly to the Linux file commands, and serve as your primary interface with HDFS. We are going to have a look into HDFS by interacting with it from the command line. We will take a look at the most common file management tasks in Hadoop, which include:

- Adding files and directories to HDFS
- Retrieving files from HDFS to local file
- system Deleting files from HDFS

ALGORITHM:**SYNTAX AND COMMANDS TO ADD, RETRIEVE AND DELETE DATA FROM HDFS****Step-1****Adding Files and Directories to HDFS**

Before you can run Hadoop programs on data stored in HDFS, you will need to put the data into HDFS first. Let's create a directory and put a file in it. HDFS has a default working directory of /user/\$USER, where \$USER is your login user name. This directory is not automatically created

for you, though, so let's create it with the `mkdir` command. For the purpose of illustration, we use chuck. You should substitute your user name in the example commands.

```
hadoop fs -mkdir /user/chuck
```

```
hadoop fs -put example.txt
```

```
hadoop fs -put example.txt /user/chuck
```

Step-2

Retrieving Files from HDFS

The Hadoop command `get` copies files from HDFS back to the local file system. To retrieve `example.txt`, we can run the following command:

```
hadoop fs -cat example.txt
```

Step-3

Deleting Files from HDFS

```
hadoop fs -rm example.txt
```

- Command for creating a directory in hdfs is “**hdfs dfs -mkdir /lendicse**”.
- Adding directory is done through the command “**hdfs dfs -put lendi_english /**”.

Step-4

Copying Data from NFS to HDFS

Copying from directory command is “**hdfs dfs -copy From Local**

/home/lendi/Desktop/shakes/glossary /lendicse/”

- View the file by using the command “**hdfs dfs -cat /lendi_english/glossary**”
- Command for listing of items in Hadoop is “**hdfs dfs -ls hdfs://localhost:9000/**”.
- Command for Deleting files is “**hdfs dfs -rm r /kartheek**”.

SAMPLE INPUT:

Input as any data format of type structured, Unstructured or Semi Structured

EXPECTED OUTPUT:

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
drwxr-xr-x	lendi	supergroup	0 B	Wed 17 Aug 2016 02:44:00 AM EDT	0	0 B	lendi_english
drwxr-xr-x	lendi	supergroup	0 B	Wed 17 Aug 2016 02:17:48 AM EDT	0	0 B	sadhana
drwxr-xr-x	lendi	supergroup	0 B	Sat 13 Aug 2016 01:31:42 AM EDT	0	0 B	shakes
drwxr-xr-x	lendi	supergroup	0 B	Sat 13 Aug 2016 01:35:59 AM EDT	0	0 B	shakes1
drwx-----	lendi	supergroup	0 B	Sat 13 Aug 2016 01:19:03 AM EDT	0	0 B	tmp

RESULT:

Thus the various file management

Ex No.	3	DEVELOP A MAPREDUCE PROGRAM TO PERFORM WORD COUNT ANALYSIS ON A TEXT DATASET
Date		

AIM:**Aim**

To develop a **MapReduce program** using Python to perform **Word Count analysis** on a text dataset by counting the frequency of each word using the Hadoop MapReduce framework.

DESCRIPTION:

MapReduce is a programming model and processing framework in **Apache Hadoop** used to process and analyze large volumes of data in a distributed computing environment. It divides a large task into smaller sub-tasks, processes them in parallel across multiple nodes, and combines the results to produce the final output.

Components of MapReduce**1. Input**

The input data is stored in the **Hadoop Distributed File System (HDFS)**. Large files are divided into smaller blocks and distributed across different nodes in the Hadoop cluster.

2. Mapper

The **Mapper** is the first phase of MapReduce. It reads the input data and converts it into intermediate **key-value pairs**.

Functions of Mapper:

- Reads the input data.
- Processes each record.
- Generates intermediate key-value pairs.

3. Shuffle and Sort

The **Shuffle and Sort** phase is automatically performed by Hadoop between the Mapper and Reducer.

Functions:

- Groups all identical keys together.
- Sorts the keys alphabetically.

- Transfers grouped data to the appropriate Reducer.

4. Reducer

The **Reducer** processes the grouped key-value pairs received from the Shuffle and Sort phase. It performs aggregation, summation, filtering, or other computations to produce the final result.

ALGORITHM

MAPPER ALGORITHM

Step 1: Start the Mapper program.

Step 2: Import the required Python module (`sys`) to read input from standard input.

Step 3: Read each line from the input text file.

Step 4: Remove leading and trailing whitespace using the `strip()` function.

Step 5: Split the line into individual words using the `split()` function.

Step 6: Convert each word to lowercase (optional) to avoid treating uppercase and lowercase words as different words.

Step 7: Emit each word as the **key** and the value **1**.

Step 8: Repeat Steps 3–7 until all lines in the input file are processed.

SHUFFLE AND SORT PHASE (PERFORMED BY HADOOP)

Step 1: Collect all key-value pairs generated by the Mapper.

Step 2: Group identical words together.

Step 3: Sort the words alphabetically.

Step 4: Forward the grouped data to the Reducer.

REDUCER ALGORITHM

Step 1: Start the Reducer program.

Step 2: Read the grouped key-value pairs received from Hadoop.

Step 3: Initialize variables to store the current word and its count.

Step 4: Compare the current word with the previous word.

Step 5: If the word is the same, add the count to the current total.

Step 6: If a new word is encountered:

- Display the previous word and its total count.
- Reset the counter for the new word.

Step 7: Repeat Steps 2–6 until all words are processed.

Step 8: Display the count of the last word.

PROGRAM

Mapper Program (mapper.py)

```
import sys

for line in sys.stdin:
    words = line.strip().split()

    for word in words:
        word = word.lower()
        print(f"{word}\t1")
```

Reducer Program (reducer.py)

```
import sys

current_word = None
current_count = 0

for line in sys.stdin:
    word, count = line.strip().split("\t")
    count = int(count)

    if current_word == word:
        current_count += count
    else:
        if current_word is not None:
            print(f"{current_word}\t{current_count}")

        current_word = word
        current_count = count

if current_word is not None:
    print(f"{current_word}\t{current_count}")
```

SAMPLE INPUT (INPUT.TXT)

Hadoop is a Big Data framework
Hadoop uses MapReduce
Python can be used with Hadoop Streaming
MapReduce processes Big Data efficiently

OUTPUT

a	1
be	1
big	2
data	2
efficiently	1
framework	1
hadoop	3
is	1
mapreduce	2
processes	1
python	1
streaming	1
used	1
uses	1
with	1
can	1

RESULT:

Thus the Word Count Map Reduce program has been implemented.

Ex No.	4	IMPLEMENT HADOOP STREAMING USING PYTHON SCRIPTS FOR SIMPLE DATA PROCESSING TASKS
Date		

AIM

To implement Hadoop Streaming using Python Mapper and Reducer scripts to process student records and calculate the **department-wise average marks** using the Hadoop MapReduce framework.

DESCRIPTION:

Hadoop Streaming is a utility provided by Apache Hadoop that enables users to create and execute **MapReduce applications using scripting languages** such as **Python, Perl, Ruby, Shell, and PHP**, instead of Java. It allows any executable or script that can read input from **standard input (stdin)** and write output to **standard output (stdout)** to function as a Mapper or Reducer.

In Hadoop Streaming, the **Mapper** processes the input data and generates intermediate key-value pairs, while the **Reducer** receives the grouped and sorted key-value pairs from Hadoop and performs aggregation or other processing to produce the final output. Hadoop automatically handles the **Shuffle and Sort** phase between the Mapper and Reducer.

ALGORITHM:

Mapper Algorithm

Step 1: Start the Mapper program.

Step 2: Import the required Python module (`sys`) to read input from the standard input stream.

Step 3: Read each line (student record) from the input file.

Step 4: Remove any leading or trailing white spaces from the input line using the `strip()` function.

Step 5: Check whether the line is not empty.

Step 6: Split the input record using a comma (,) as the delimiter.

Step 7: Verify that the record contains exactly three fields.

Step 8: Store the values in separate variables.

Step 9: Emit the department as the **key** and marks as the **value**.

Step 10: Repeat Steps 3–9 until all records are processed.

Step 11: Stop the Mapper.

Hadoop Shuffle and Sort Phase

Step 1: After the Mapper finishes execution, Hadoop automatically performs the **Shuffle and Sort** operation.

During this phase:

- Records having the same department are grouped together.
- Keys are sorted alphabetically.
- All marks belonging to the same department are sent to one Reducer.

Reducer Algorithm

Step 1: Start the Reducer program.

Step 2: Import the `sys` module.

Step 3: Read each key-value pair received from Hadoop.

Step 4: Split the input using the tab (`\t`) delimiter.

Step 5: Convert the marks into an integer and also Check whether the current department is the same as the previous department.

Step 6: If a new department is encountered, calculate the average of the previous department.

Step 7: Reset the variables for the new department.

Step 8 : Repeat the Steps 4–7 until all records are processed.

PROGRAM: mapper.py

```
import sys
```

```
for line in sys.stdin:
```

```
    line = line.strip()
```

```
    if line:
```

```
fields = line.split(",")
```

```
if len(fields) == 3:
```

```
    name, dept, marks = fields
```

```
    print(f"{dept}\t{marks}")
```

reducer.py

```
import sys
```

```
current_dept = None
```

```
total_marks = 0
```

```
student_count = 0
```

```
for line in sys.stdin:
```

```
    dept, marks = line.strip().split("\t")
```

```
    marks = int(marks)
```

```
    if current_dept == dept:
```

```
        total_marks += marks
```

```
        student_count += 1
```

```
    else:
```

```
        if current_dept is not None:
```

```
            average = total_marks / student_count
```

```
            print(f"{current_dept}\tAverage = {average:.2f}")
```

```
        current_dept = dept
```

```
        total_marks = marks
```

```
        student_count = 1
```

```
if current_dept is not None:
```

```
    average = total_marks / student_count
```

```
print(f'{current_dept}\tAverage = {average:.2f}')
```

Input File

students.txt

```
Arun,CSE,82
Priya,AIML,91
Karthik,CSE,75
Divya,AIML,88
Rahul,IT,79
Meena,CSE,95
Siva,IT,85
Anitha,AIML,94
```

OUTPUT

```
hadoop jar %HADOOP_HOME%\share\hadoop\tools\lib\hadoop-streaming-*.jar ^
-input /students/students.txt ^
-output /students/output ^
-mapper "python mapper.py" ^
-reducer "python reducer.py" ^
-file mapper.py ^
-file reducer.py
```

MAPPED OUTPUT

```
CSE  82
AIML  91
CSE  75
AIML  88
IT    79
CSE  95
IT    85
AIML  94
```

REDUCED OUTPUT

AIML Average = 91.00

CSE Average = 84.00

IT Average = 82.00

RESULT

Thus, the Hadoop Streaming application was successfully implemented using Python scripts to calculate the **department-wise average marks** of students. The Mapper grouped records by department, and the Reducer computed the average marks for each department, demonstrating aggregation and analysis using the Hadoop MapReduce framework.

Ex No.	5	Perform data storage and retrieval operations using Cassandra Query Language (CQL).
Date		

AIM

To perform **data storage and retrieval operations** using **Cassandra Query Language (CQL)** by creating a keyspace and table, inserting records, and executing various retrieval queries in Apache Cassandra.

DESCRIPTION:

Cassandra Query Language (CQL) is the standard query language used to interact with the **Apache Cassandra** NoSQL database. It provides an SQL-like syntax that simplifies database operations such as creating keyspaces and tables, inserting, updating, deleting, and retrieving data. Although its syntax resembles SQL, CQL is designed specifically for Cassandra's distributed architecture and column-family data model. It enables efficient management of large-scale, distributed datasets while ensuring high availability, scalability, and fault tolerance.

Key Features of CQL:

- SQL-like syntax, making it easy to learn and use.
- Supports **CREATE**, **INSERT**, **SELECT**, **UPDATE**, and **DELETE** operations.
- Organizes data into **keyspaces**, **tables**, **rows**, and **columns**.
- Optimized for distributed storage and fast data retrieval.
- Supports high scalability and fault-tolerant data management in Apache Cassandra.

Open and Execute Apache Cassandra (Windows)

Requirement to start Cassandra:

- Java JDK 8 or later installed
- JAVA_HOME environment variable configured
- Apache Cassandra downloaded and extracted (e.g., C:\apache-cassandra-4.1.x)

Step 1: Open Command Prompt Press Win + R, type: **cmd** and press Enter.

Step 2: Navigate to the Cassandra Bin Folder **>cd C:\apache-cassandra-4.1.8\bin** (Replace 4.1.8 with your installed version.)

Step 3: Start Cassandra Server **>cassandra.bat**

If Cassandra starts successfully, then the log messages similar to:

Starting listening for CQL clients on localhost/127.0.0.1:9042...

Step 4: Open a **New Command Prompt** type in windows command prompt **cmd**. Open another Command Prompt and navigate to the same bin directory:

>**cd C:\apache-cassandra-4.1.8\bin**

Step 5: Open CQL Shell >**cqlsh**

After successful completion it will display the following message in command prompt: >**cqlsh**

ALGORITHM:

Step 1: Start Cassandra

1. Launch Apache Cassandra.
2. Open the CQL Shell (cqlsh).

Step 2: Create a Keyspace

1. Create a keyspace named StudentDB.
2. Specify the replication strategy.
3. Activate the keyspace using the `USE` command.

Step 3: Create a Table, Insert, Retrieve, Update, Delete and Display the Records

Execute CQL Commands:

Create Keyspace

```
CREATE KEYSPACE StudentDB
WITH replication = {
'class': 'SimpleStrategy',
'replication_factor': 1
};
```

Use Keyspace

```
USE StudentDB;
```

Create Table

```
CREATE TABLE Student( Student_ID INT PRIMARY KEY,
Name TEXT,
Department TEXT,
Year INT,
Marks INT );
```

Insert Records

```
INSERT INTO Student(Student_ID, Name, Department, Year, Marks) VALUES(101,'Arun','CSE',3,82);
```

```
INSERT INTO Student(Student_ID, Name, Department, Year, Marks) VALUES(102,'Priya','AIML',3,91);
```

```
INSERT INTO Student(Student_ID, Name, Department, Year, Marks) VALUES(103,'Rahul','IT',3,79);
```

```
INSERT INTO Student(Student_ID, Name, Department, Year, Marks) VALUES(104,'Divya','AIML',3,88);
```

```
INSERT INTO Student(Student_ID, Name, Department, Year, Marks) VALUES(105,'Meena','CSE',3,95);
```

Display All Records

```
SELECT * FROM Student;
```

Student_ID	Name	Department	Year	Marks
101	Arun	CSE	3	82
102	Priya	AIML	3	91
103	Rahul	IT	3	79
104	Divya	AIML	3	88
105	Meena	CSE	3	95

Retrieve a Particular Student

```
SELECT * FROM Student WHERE Student_ID = 102;
```

Student_ID	Name	Department	Year	Marks
102	Priya	AIML	3	91

Retrieve Selected Columns

```
SELECT Name, Department FROM Student;
```

Retrieve Students with ALLOW FILTERING

```
SELECT *
```

```
FROM Student
```

```
WHERE Department='AIML'
```

```
ALLOW FILTERING;
```

Student_ID	Name	Department	Year	Marks
102	Priya	AIML	3	91
104	Divya	AIML	3	88

Update a Record

```
UPDATE Student SET Marks = 94 WHERE Student_ID = 102;
```

Verify Updated Record

SELECT * FROM Student WHERE Student_ID = 102;

Student_ID	Name	Department	Year	Marks
102	Priya	AIML	3	94

Delete a Record

DELETE FROM Student WHERE Student_ID = 103;

Verify Deletion

SELECT * FROM Student;

Student_ID	Name	Department	Year	Marks
101	Arun	CSE	3	82
102	Priya	AIML	3	94
104	Divya	AIML	3	88
105	Meena	CSE	3	95

RESULT

Thus, the **data storage and retrieval operations** were successfully performed using **Cassandra Query Language (CQL)**. A keyspace and table were created, student records were inserted, retrieved, updated, and deleted successfully. The experiment demonstrated the use of CQL commands for efficient data management in the Apache Cassandra NoSQL database.

Ex No.	6	CREATE AND MANIPULATE KEY-VALUE AND DOCUMENT-ORIENTED DATA MODELS USING NOSQL DATABASES
Date		

AIM

To create and manipulate **Key-Value** and **Document-Oriented** data models using Python by performing **Create, Read, Update, and Delete (CRUD)** operations on NoSQL data structures.

DESCRIPTION:

NoSQL (Not Only SQL) databases are widely used in Big Data applications because they provide flexible schemas, high scalability, and efficient storage of structured, semi-structured, and unstructured data. Python provides built-in data structures that can simulate NoSQL data models.

- **Key-Value Data Model:** Implemented using a Python dictionary, where each key uniquely identifies a value. It is suitable for fast data retrieval and caching.
- **Document-Oriented Data Model:** Implemented using a list of dictionaries, where each dictionary represents a JSON-like document containing multiple fields. This model is commonly used in applications such as e-commerce, healthcare, and social media.

This experiment demonstrates CRUD operations on both data models using Python.

ALGORITHM:

Part A: Key-Value Data Model

Step 1: Start the Python program.

Step 2: Create a Python dictionary to store product details with Product ID as the key and Product Name as the value.

Step 3: Insert multiple key-value pairs into the dictionary.

Step 4: Display all stored products.

Step 5: Retrieve a product using its Product ID.

Step 6: Update the value of an existing key.

Step 7: Insert a new key-value pair.

Step 8: Delete an existing key-value pair.

Step 9: Display the updated dictionary.

Step 10: Terminate the program.

Part B: Document-Oriented Data Model

Step 1: Create a list containing multiple product documents.

Step 2: Store each product as a Python dictionary.

Step 3: Display all product documents.

Step 4: Retrieve products based on a specified category.

Step 5: Update the price of a selected product.

Step 6: Insert a new product document.

Step 7: Delete a product document.

Step 8: Display the updated collection.

Step 9: Terminate the program.

PROGRAM

Part A: Key-Value Data Model

```
print("==== KEY-VALUE DATA MODEL ====")
```

```
# Create Key-Value Store
```

```
products = {
    "P101": "Laptop",
    "P102": "Smartphone",
    "P103": "Headphones"
}
```

```
# Display Products
```

```
print("\nAvailable Products")
for key, value in products.items():
    print(key, ":", value)
```

```
# Retrieve Product
```

```
print("\nRetrieve Product P102")
print(products["P102"])
```

```
# Update Product
```

```
products["P102"] = "5G Smartphone"
```

```
print("\nAfter Updating Product")
```

```
for key, value in products.items():
    print(key, ":", value)
```

```
# Add New Product
products["P104"] = "Tablet"

print("\nAfter Adding Product")
for key, value in products.items():
    print(key, ":", value)
```

```
# Delete Product
del products["P103"]

print("\nAfter Deleting Product")
for key, value in products.items():
    print(key, ":", value)
```

Part B: Document-Oriented Data Model

```
print("\n===== DOCUMENT-ORIENTED DATA MODEL =====")
```

```
products = [
    {
        "ProductID": "P101",
        "ProductName": "Laptop",
        "Category": "Electronics",
        "Price": 65000,
        "Stock": 20
    },
    {
        "ProductID": "P102",
        "ProductName": "Smartphone",
        "Category": "Electronics",
        "Price": 28000,
        "Stock": 35
    },
    {
        "ProductID": "P103",
        "ProductName": "Office Chair",
        "Category": "Furniture",
        "Price": 7500,
        "Stock": 15
    }
]
```

```
# Display All Products
print("\nAll Products")
for product in products:
    print(product)
```

```

# Retrieve Electronics Products
print("\nElectronics Products")
for product in products:
    if product["Category"] == "Electronics":
        print(product)

# Update Product Price
for product in products:
    if product["ProductID"] == "P102":
        product["Price"] = 26000

print("\nAfter Updating Price")
for product in products:
    print(product)

# Add New Product
products.append({
    "ProductID": "P104",
    "ProductName": "Tablet",
    "Category": "Electronics",
    "Price": 22000,
    "Stock": 18
})

print("\nAfter Adding Product")
for product in products:
    print(product)

# Delete Product
products = [p for p in products if p["ProductID"] != "P103"]

print("\nAfter Deleting Product")
for product in products:
    print(product)

```

SAMPLE OUTPUT

===== KEY-VALUE DATA MODEL =====

Available Products
P101 : Laptop
P102 : Smartphone
P103 : Headphones

Retrieve Product P102
Smartphone

After Updating Product

P101 : Laptop

P102 : 5G Smartphone

P103 : Headphones

After Adding Product

P101 : Laptop

P102 : 5G Smartphone

P103 : Headphones

P104 : Tablet

After Deleting Product

P101 : Laptop

P102 : 5G Smartphone

P104 : Tablet

===== DOCUMENT-ORIENTED DATA MODEL =====

All Products

```
{'ProductID': 'P101', 'ProductName': 'Laptop', 'Category': 'Electronics', 'Price': 65000, 'Stock': 20}
```

```
{'ProductID': 'P102', 'ProductName': 'Smartphone', 'Category': 'Electronics', 'Price': 28000, 'Stock': 35}
```

```
{'ProductID': 'P103', 'ProductName': 'Office Chair', 'Category': 'Furniture', 'Price': 7500, 'Stock': 15}
```

Electronics Products

```
{'ProductID': 'P101', 'ProductName': 'Laptop', 'Category': 'Electronics', 'Price': 65000, 'Stock': 20}
```

```
{'ProductID': 'P102', 'ProductName': 'Smartphone', 'Category': 'Electronics', 'Price': 28000, 'Stock': 35}
```

After Updating Price

```
{'ProductID': 'P102', 'ProductName': 'Smartphone', 'Category': 'Electronics', 'Price': 26000, 'Stock': 35}
```

After Adding Product

```
{'ProductID': 'P104', 'ProductName': 'Tablet', 'Category': 'Electronics', 'Price': 22000, 'Stock': 18}
```

After Deleting Product

```
{'ProductID': 'P101', 'ProductName': 'Laptop', 'Category': 'Electronics', 'Price': 65000, 'Stock': 20}
```

```
{'ProductID': 'P102', 'ProductName': 'Smartphone', 'Category': 'Electronics', 'Price': 26000, 'Stock': 35}
```

```
{'ProductID': 'P104', 'ProductName': 'Tablet', 'Category': 'Electronics', 'Price': 22000, 'Stock': 18}
```

RESULT

Thus, the Key-Value and Document-Oriented data models were successfully created and manipulated using Python. CRUD operations such as Create, Read, Update, and Delete were performed successfully using Python dictionaries and lists of dictionaries, demonstrating the concepts of NoSQL data models commonly used in Big Data applications.